

The process of producing a binary executable from source code involves several stages, each performed by different tools. The main stages include compilation, assembly, and linking. Here's an overview of the tasks performed by a compiler, an assembler, and a linker in producing a binary executable:

1. Compiler:

Task:

- The compiler translates high-level source code written in a programming language (like C, C++, etc.) into an intermediate representation or machine code specific to the target architecture.

Steps:

- **Lexical Analysis (Scanning):**
 - Breaks the source code into tokens (keywords, identifiers, operators).
- **Syntax Analysis (Parsing):**
 - Analyzes the structure of the code to create an abstract syntax tree (AST).
- **Semantic Analysis:**
 - Checks for semantic errors and builds a symbol table.
- **Intermediate Code Generation:**
 - Translates the source code into an intermediate code.
- **Optimization:**
 - Applies various optimizations to improve the efficiency of the code.
- **Code Generation:**
 - Converts the intermediate code into machine code or assembly code.

2. Assembler:

Task:

- The assembler translates assembly language code into machine code (binary code) that can be directly executed by the computer's CPU.

Steps:

- **Assembly Code Parsing:**
 - Reads the assembly language code line by line.
- **Symbol Resolution:**
 - Resolves symbolic references to memory addresses.
- **Translation to Machine Code:**
 - Converts assembly instructions into binary machine code.
- **Object File Generation:**
 - Produces an object file containing the machine code and additional information.

3. Linker:

Task:

- The linker combines object files and resolves external references to create a single executable file.

Steps:

- **Object File Linking:**
 - Combines multiple object files generated by the assembler into a single executable file.
- **Address Binding:**
 - Resolves addresses and assigns final memory addresses to variables and functions.
- **Symbol Resolution:**
 - Resolves references to external symbols.
- **Relocation:**
 - Adjusts addresses in the code and data sections to reflect the final memory layout.
- **Executable File Generation:**
 - Produces the final executable binary file that can be loaded into memory and executed by the computer.

Overall Workflow:

- **Compilation:**
 - The compiler translates high-level code into assembly or machine code and produces one or more object files.
- **Assembly:**
 - The assembler translates assembly code into machine code and produces additional object files.
- **Linking:**
 - The linker combines object files, resolves references, and produces the final executable binary file.